Attention & Transformers

What's In the News?

Just now, NVIDIA revolutionized itself: The intelligent agent evolved autonomously for 7 days, eliminating all operator engineers and GPU experts.

机器之心 2026-03-26 00:10

Oh no, human cognition is the bottleneck.

CNET YOUR GUIDE TO A BETTER FUTURE

News AI Tech Home Entertainment Wellness Gif

ai atlas > Tech > Services & Software > AI

Anthropic's Claude Can Now Control Your Computer

The developer is boosting Claude's agentic capabilities in an apparent effort to compete with platforms like OpenClaw.

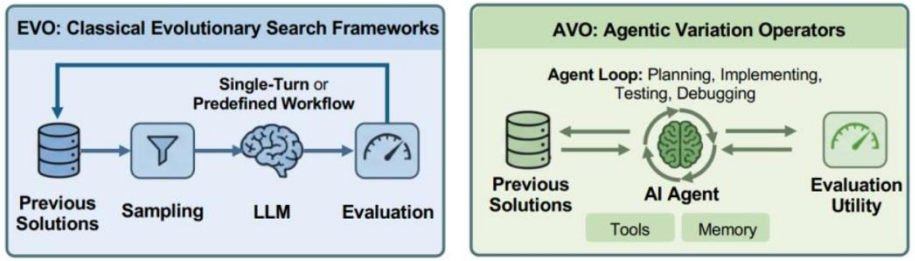


Figure 1: **EVO vs AVO**: Comparison between prior evolutionary search frameworks (e.g. FunSearch, AlphaEvolve, and related LLM-augmented evolutionary approaches) and the proposed Agentic Variation Operator. **Left:** Prior approaches follow a fixed pipeline where the LLM is confined to a single-turn generation step or a predefined workflow, with sampling and evaluation controlled by the framework. **Right:** AVO replaces this pipeline with an autonomous AI agent that iteratively plans, implements, tests, and debugs across long-running sessions, with direct access to previous solutions, evaluation utilities, tools, and persistent memory.

To verify its effectiveness, NVIDIA applied AVO to the multi-head attention (MHA) kernel on the **NVIDIA Blackwell B200 GPU** and directly compared it with the expert-optimized cuDNN and FlashAttention-4 kernels. In a continuous 7-day autonomous evolution without human intervention, the agent explored more than 500 optimization directions and evolved 40 kernel versions. The final generated MHA kernel achieved a maximum throughput of **1668 TFLOPS** at BF16 precision, surpassing cuDNN by up to **3.5%** and FlashAttention-4 by up to **10.5%** in the test configuration.

This Session's Agenda

Attention Mechanism in RNNs for Translation (2014)

- Sequence-to-sequence models with attention in translation tasks.

Scaled Dot Product Attention

- Hard-code a rule for determining semantic similarity between a focal token and all other tokens in the sequence

QKV Framework & Multi-head Attention

- We can incorporate trainable weights around the attention mechanism.

TF-IDF is "Static Attention"

Not all phrases are of equal importance…

- E.g., David less important than Beckham
- If a term occurs all the time, observing its presence is less informative

Inverse-document frequency (IDF) helps address this.

$$\mathrm{IDF} = \log(N/n_j)$$

- Term 'weighting' is then calculated as Term Frequency (TF) x IDF
- n_j= # of docs containing the term, N = total # of docs
- A term is deemed important if it has a high TF and/or a high IDF.
- As TF goes up, the word is more common generally. As IDF goes up, it means very few documents contain this term.

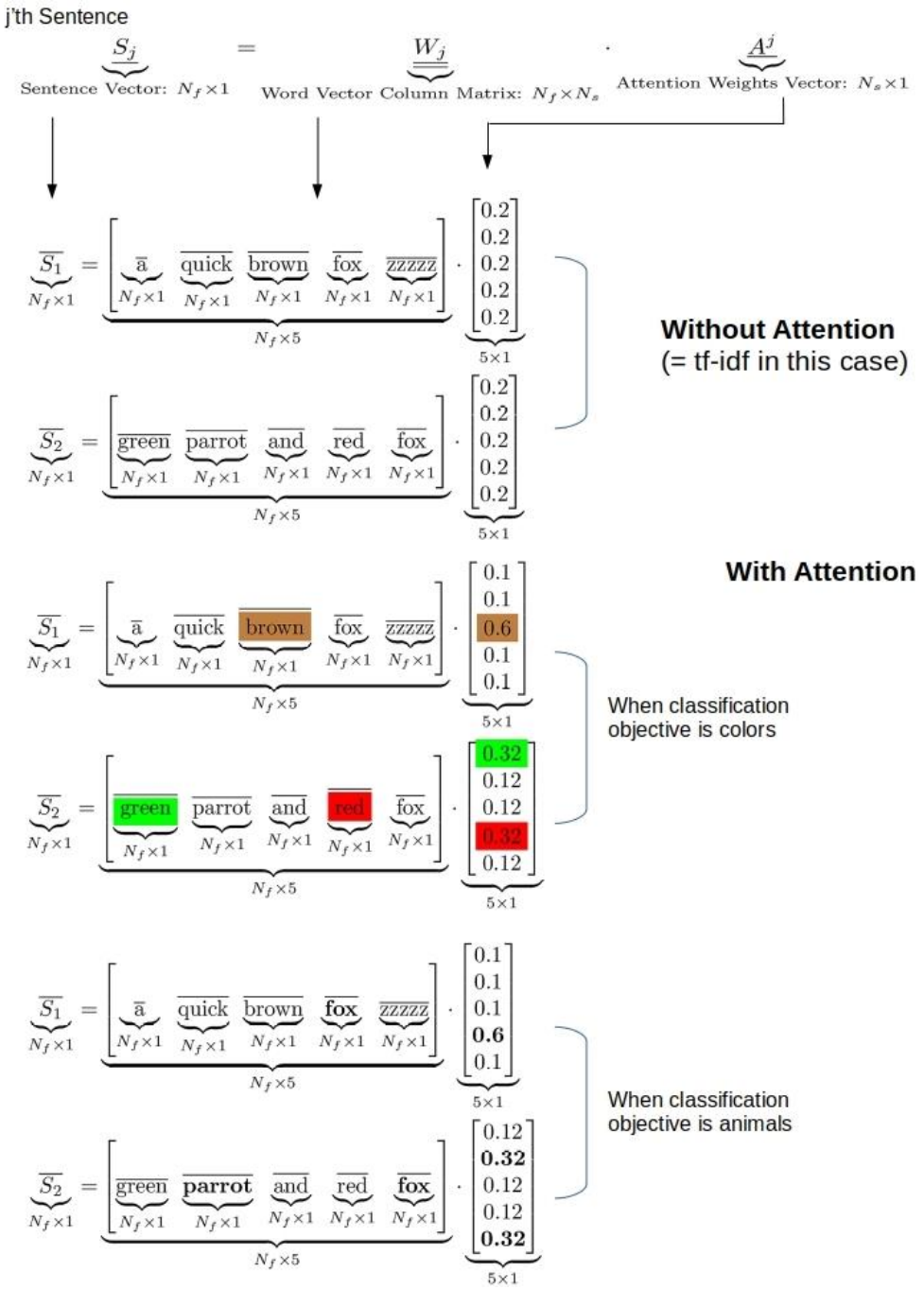
What Would Adaptive Weights Look Like?

Figure Out Weights Useful for Prediction

- Rather than telling the network the weights, which is what TF-IDF effectively does, we provide the network a weight matrix to update!

This Yields Task-specific Token Weights

- See some simple examples on the right.
- Our network may learn that certain tokens are more important for predicting a given label.



The First (RNN) Attention Model

Neural Machine Translation by Jointly Learning to Align and Translate

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

" … allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word …"

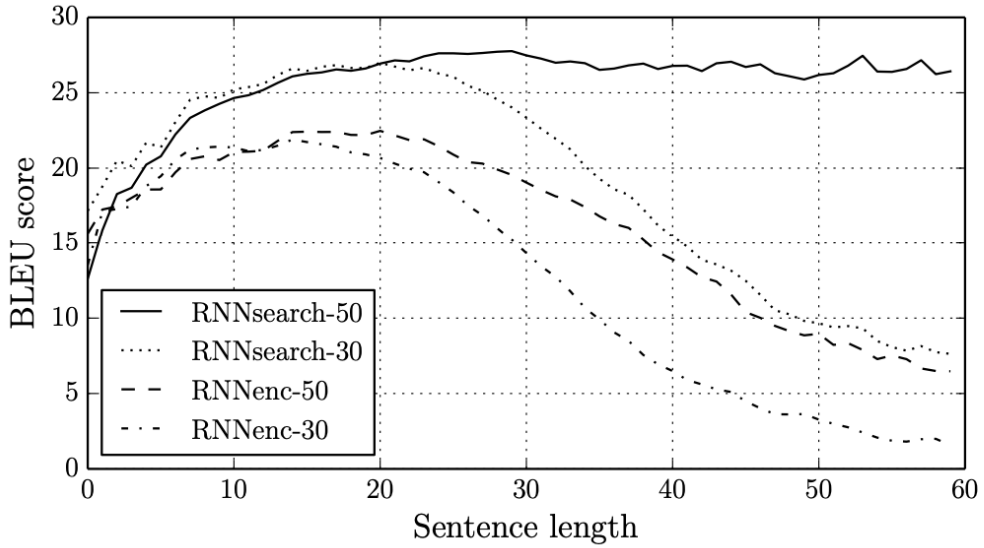
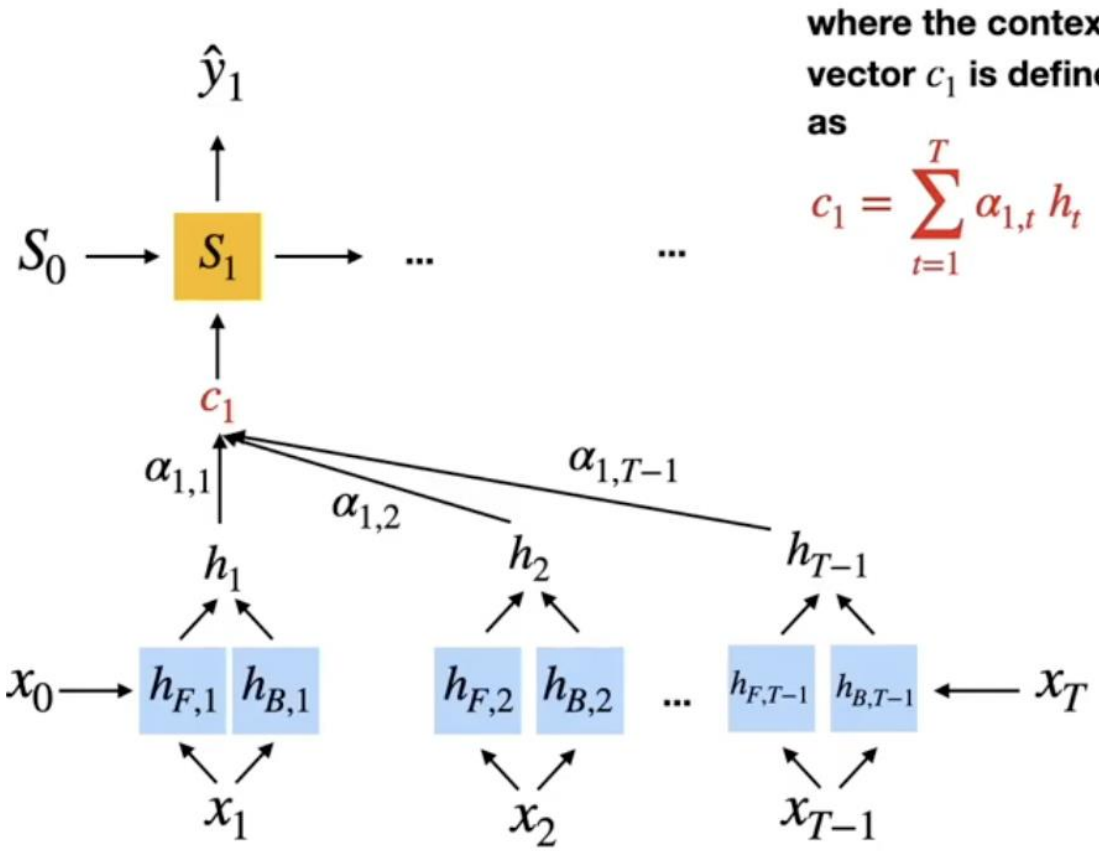


Figure 2: The BLEU scores of the generated translations on the test set with respect to the lengths of the sentences. The results are on the full test set which includes sentences having unknown words to the models.

Bahdanau (2014) Attention



where the context vector c_1 is defined as

$$c_1 = \sum_{t=1}^{T} \alpha_{1,t} h_t$$

Step by Step

1. Encode: A bidirectional RNN processes the full input sequence, producing a hidden state at every position.

2. Score: At each decoding step, compare the decoder's current hidden state to each encoder hidden state (S_t-1 vs. h's) . This produces a relevance score per input token.

3. Attention Weights: Softmax the scores (they sum to 1).

4. Context: Weighted sum of scaled encoder hidden states. This vector tells the decoder what's relevant from the rest of the sequence for the current output token.

5. Predict: Concatenate context vector with decoder hidden state predict next output token.

Computing attention weights

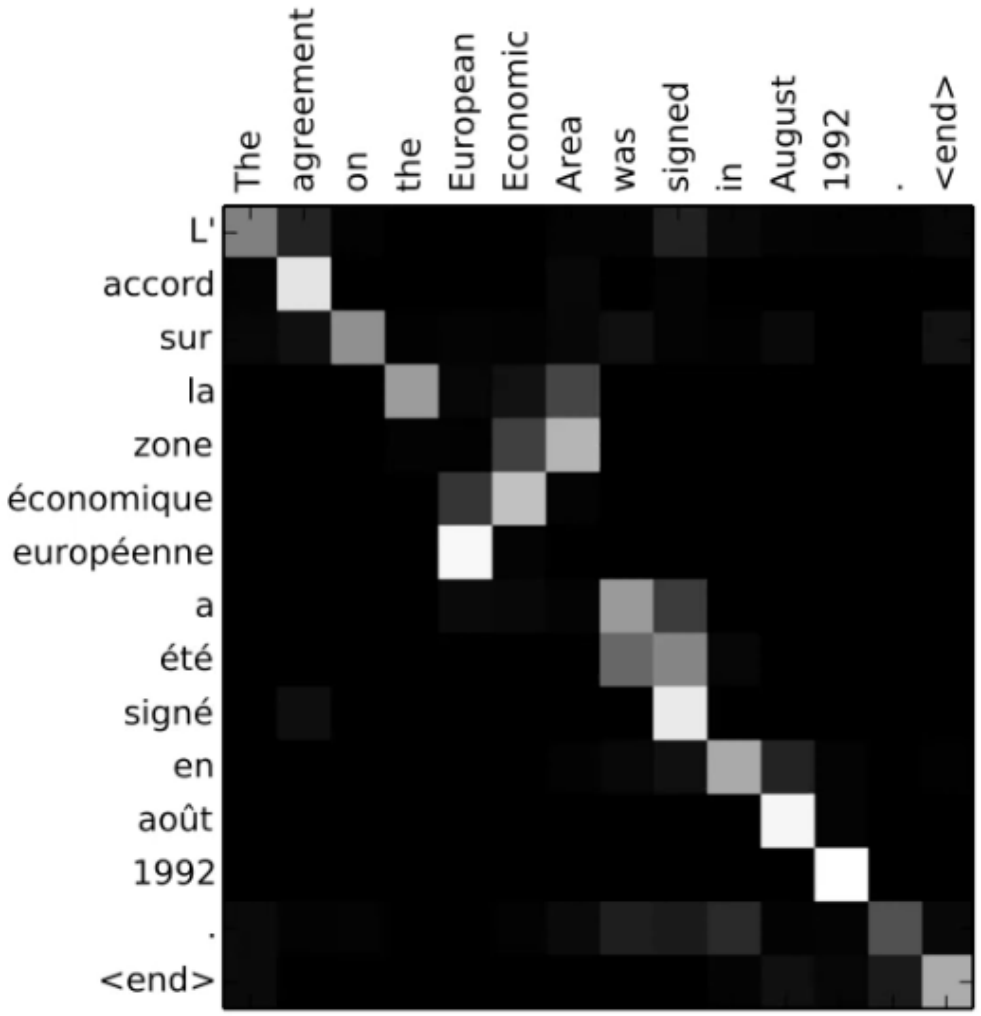
S_{t-1}, h_t → Neural Net → $e_{t,t'}$

$$\alpha_{t,t'} = \frac{\exp(e_{t,t'})}{\sum_{t'=1}^{T} \exp(e_{t,t'})}$$

Neural Net

$$= v_a^\top \tanh(W_a s_{t-1} + U_a h_i)$$

The First (RNN) Attention Model



Computing attention weights

S_{t-1}, h_t → Neural Net → $e_{t,t'}$

$$\alpha_{t,t'} = \frac{\exp(e_{t,t'})}{\sum_{t'=1}^{T} \exp(e_{t,t'})}$$

Neural Net $= v_a^\top \tanh(W_a s_{t-1} + U_a h_i)$

We Don't Need an RNN for This!

We Can Drop the RNN and Process the Whole Sequence in Parallel

- Note that transformers work with a similar encoder architecture, but no LSTMs are needed (we use "stacked attention layers").
- Transformers are more computationally expensive (more calculations required), but because we can parallelize them, they can be trained faster than RNNs.
- By moving away from RNN specific implementation we can also think about more general forms of attention (e.g.,. For CNNs).

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

A Simpler (Less-Flexible) Attention Mechanism

Main procedure:

1) Derive attention weights: similarity between current input and all other inputs (next slide)
2) Normalize weights via softmax (next slide)
3) Compute attention value from normalized weights and corresponding inputs (below)

Self-attention as weighted sum:

$$\boldsymbol{A}_i = \sum_{j=1}^{T} a_{ij}\boldsymbol{x}_j$$

output corresponding to the i-th input

weight based on similarity between current input $\boldsymbol{x}_i$ and all other inputs

A Simpler (Less-Flexible) Attention Mechanism

Self-attention as weighted sum:

$$A_i = \sum_{j=0}^{T} a_{ij} x_j$$

output corresponding to the i-th input

weight based on similarity between current input x_i and all other inputs

How to compute the attention weights?

here as simple dot product:

$$e_{ij} = x_i^\top x_j$$

repeat this for all inputs $j \in \{1...T\}$, then normalize

$$a_{ij} = \frac{\exp\left(e_{ij}\right)}{\sum_{j=1}^{T} \exp\left(e_{ij}\right)} = \text{softmax}\left(\left[e_{ij}\right]_{j=1....T}\right)$$

A Simpler (Less Flexible) Attention Mechanism

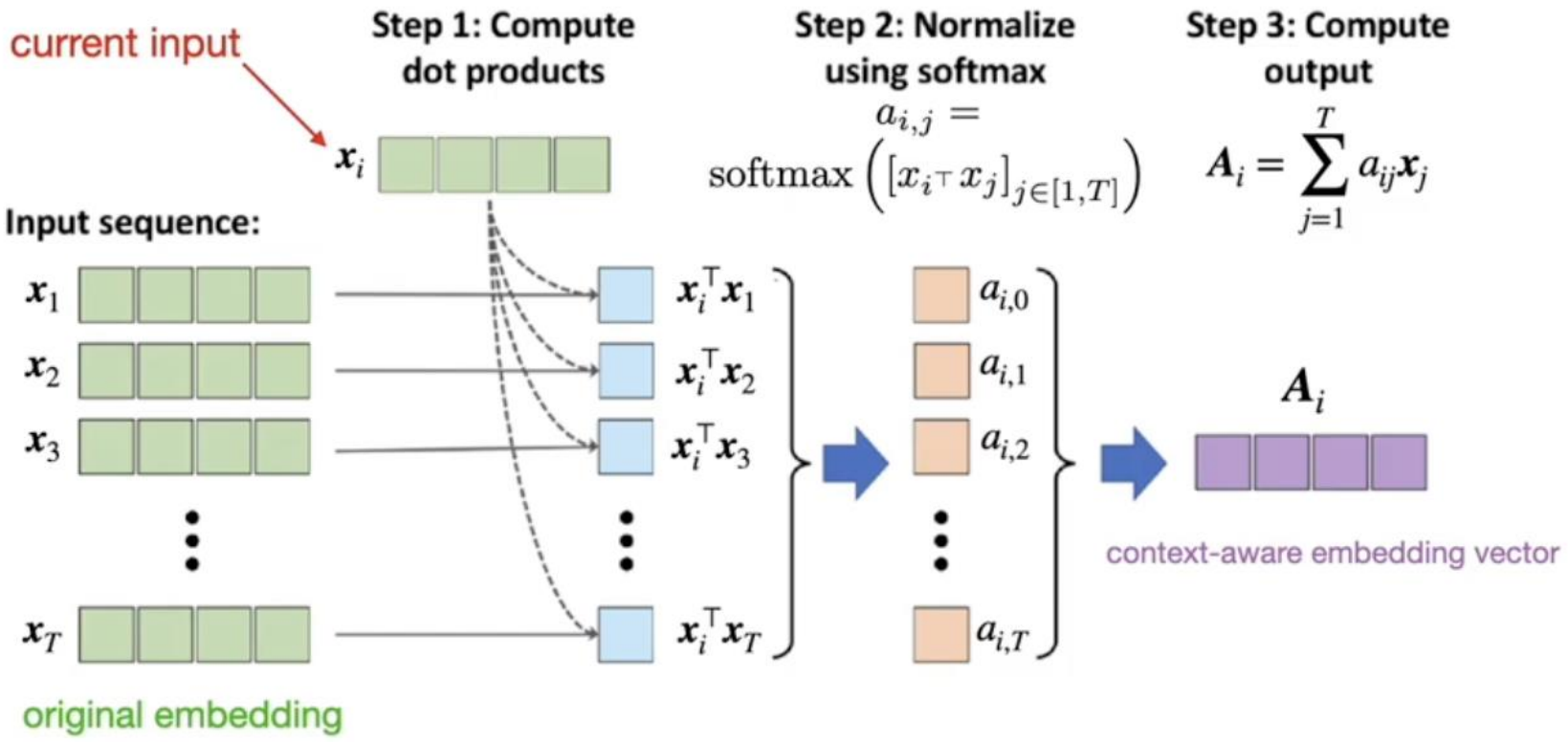
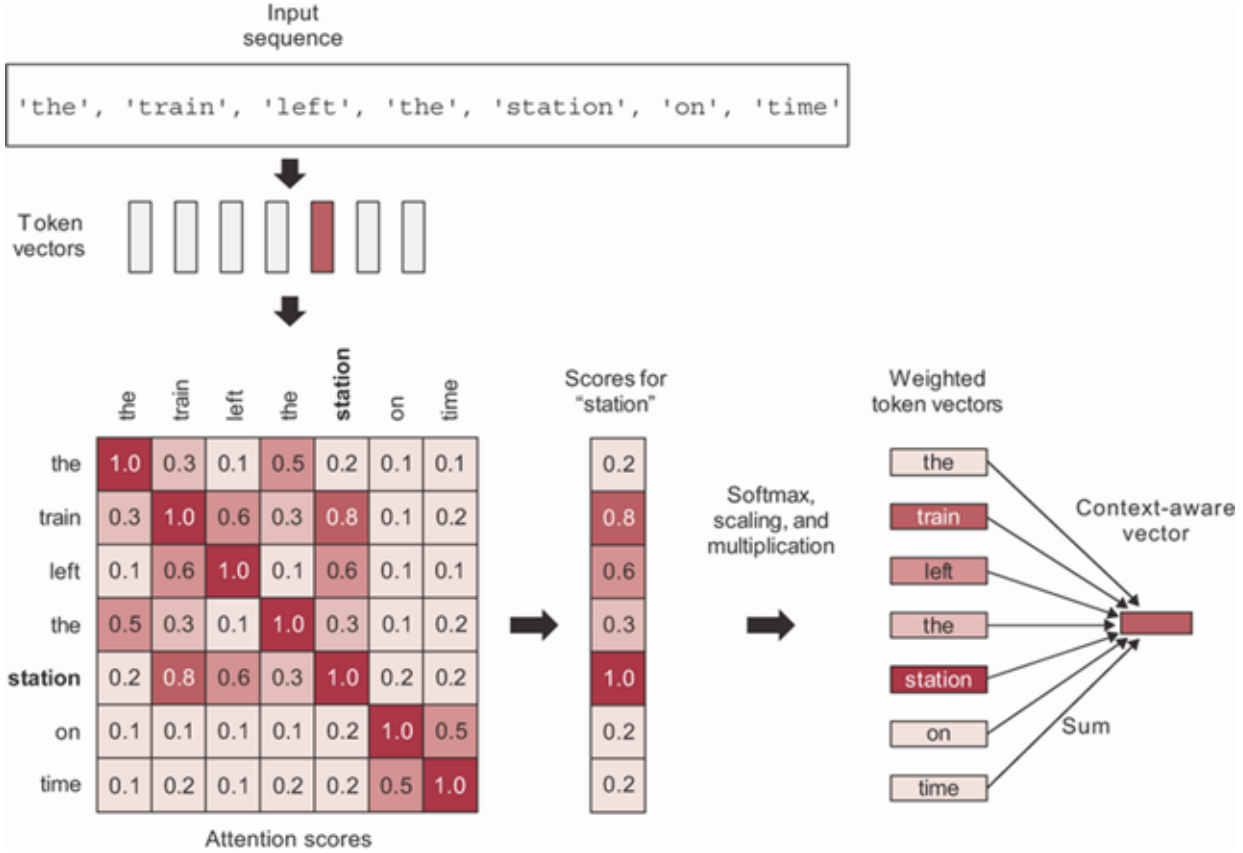


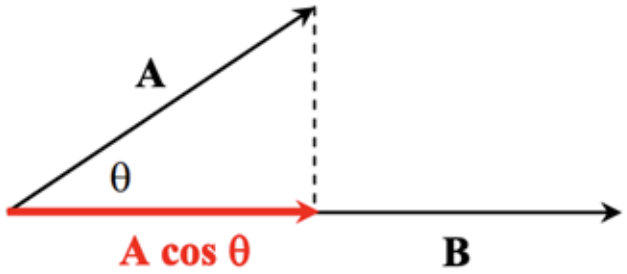
Image source: Raschka & Mirjalili 2019. Python Machine Learning, 3rd edition

keras.layers.Attention()

Basic Self-Attention

- This is what the textbook introduces.
- We ‘shift’ each input word embedding (vector representation) *toward* other words in the input that are most similar (smallest cosine distance).
- This ‘contextualizes’ each word embedding, changing its representation depending on the other words that appear alongside it.





This Attention Mechanism is 'Rigid'

The Approach Considers the Input Context in a 'Fixed' Manner

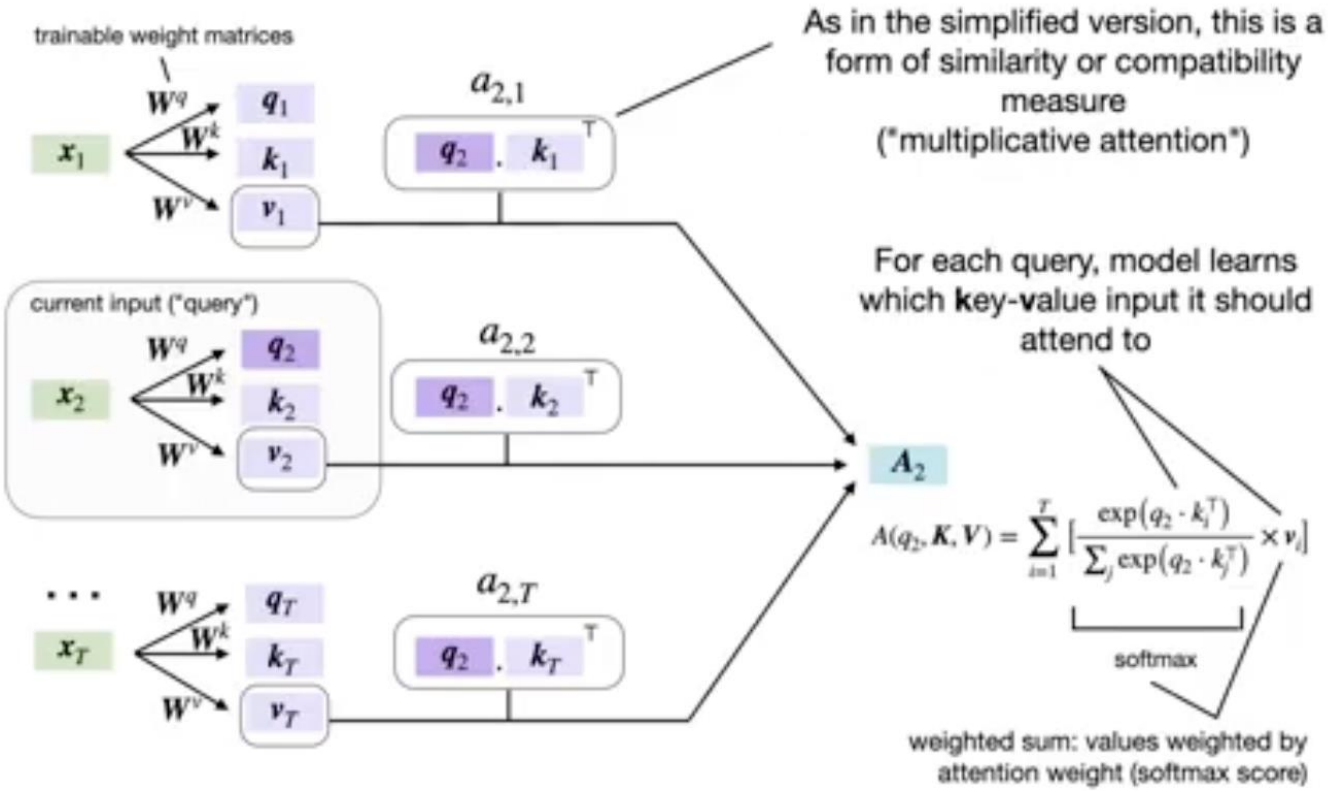
- Although it doesn't assume forward or backward sequence/ordering is most important (like an RNN does), it *does* assume a specific attention rule. **Notice the approach does not use any weights!** The network cannot 'learn' how it should decide which tokens to attend to in a context.
- We get context-specific attention weights, but the notion of how context is processed is pre-defined / hard-coded in the dot-product-summation procedure; there is no way to 'adjust' or 'adapt' this attention rule.
- This pre-defined approach to attention also might not generalize well outside of language. The pre-defined approach to attention assumes that shared semantic similarity is the best heuristic for attention, but perhaps it is not in some cases. Most obviously, perhaps in some scenarios it is ideal to focus attention on words that are semantically *dissimilar*.
- How can we enable a procedure of this sort, but injecting trainable parameters, so the network can figure out an optimal attention rule during training?

Query Key Value (QKV) Framework

We Add Some Weight Matrices!

- We begin by taking an input embedding (e.g., a word embedding) and we multiply it by a matrix of randomly initialized (but refined in training) weights.
- The NN can learn how to 'pre-transform' the embeddings, thereby modifying the overall attention rule.
- We learn to project the embedding into a 'new latent space' before doing our dot product comparisons, etc.

$$\text{query} = \boldsymbol{W}^q \boldsymbol{x}_i$$
$$\text{key} = \boldsymbol{W}^k \boldsymbol{x}_i$$
$$\text{value} = \boldsymbol{W}^v \boldsymbol{x}_i$$



‘Scaled’ Dot Product

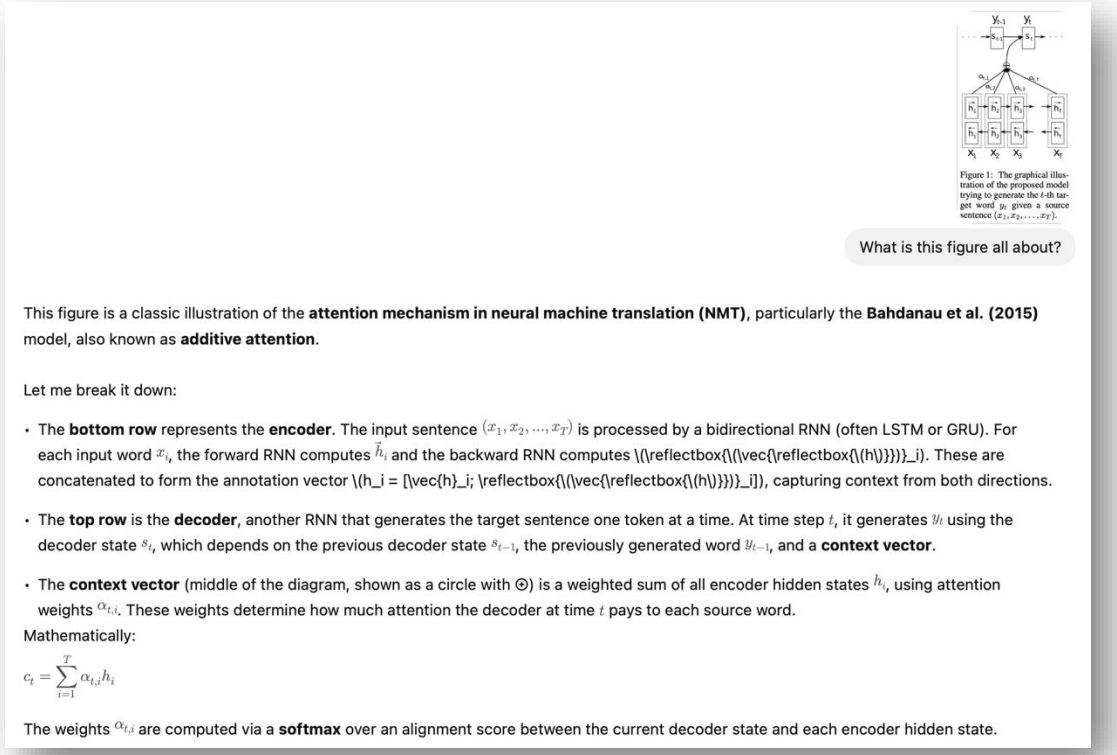
Scaled Dot-Product Attention…

- One other nuance is that we scale the Softmax input.
- This is done to prevent the dot products from getting too big, which can cause the output of softmax to tend toward 1 or 0 and lead to vanishing gradient issues.
- sqrt(d_k) is the square root of the dimensionality of the input key embeddings (in the original paper, this was the same as the dimension of the word embeddings).
- We write this in big-letter notation because we can parallelize everything as tensor calculations.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

From Self-Attention to Cross-Attention

- We now have the freedom to learn attention scores for input in one modality of input data to inform what we do with input in another modality. The dot product might be applied to compare a word embedding against a set of other document or image embeddings (this is what modern Retrieval Augmented Generation does!)



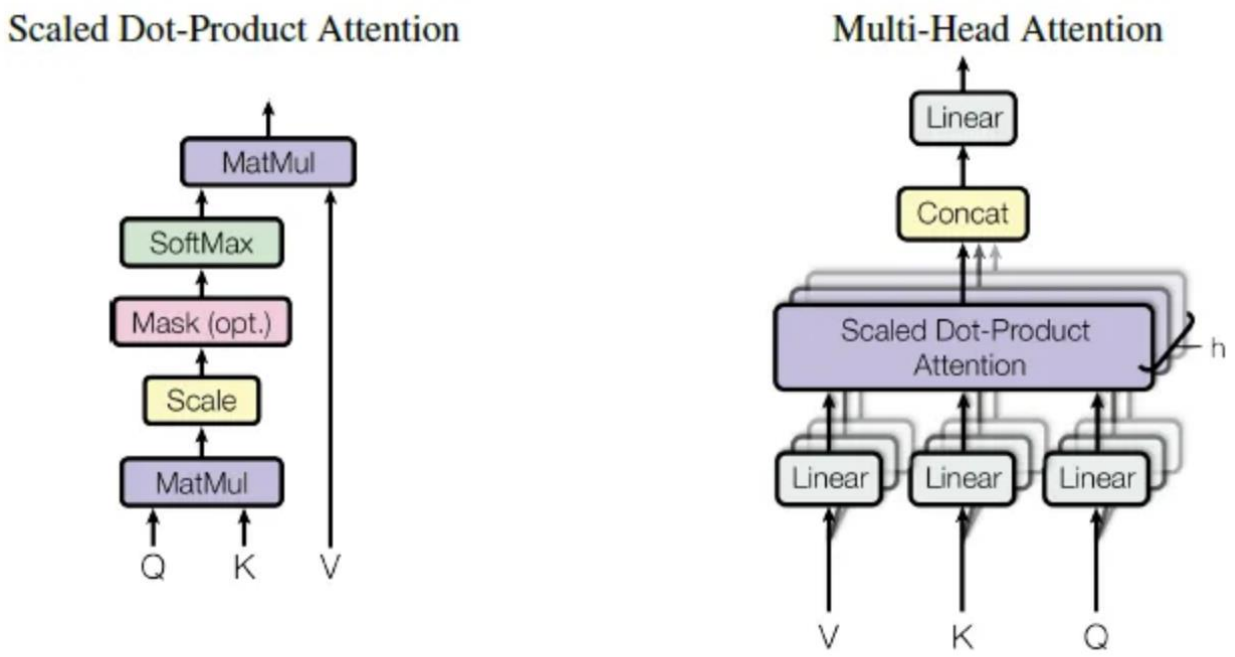
Example: Image Q&A

- You input an **image** (e.g., via a CNN or ViT), then input a **text prompt** (e.g., for captioning or VQA).
- Use **cross-attention**: the text tokens query the image features to decide what visual content is relevant before producing the textual response.

keras.layers.MultiHeadAttention()

We have Multiple 'Heads'
Paying Attention in Different Ways

- Each head can be attending to something very different.
- By using multiple sets of query, key and value weight matrices, the network can learn to consider different combinations of input features.
- This lets it capture more complex patterns in the data.



From "Attention is all you need" paper by Vaswani, et al., 2017 [1]

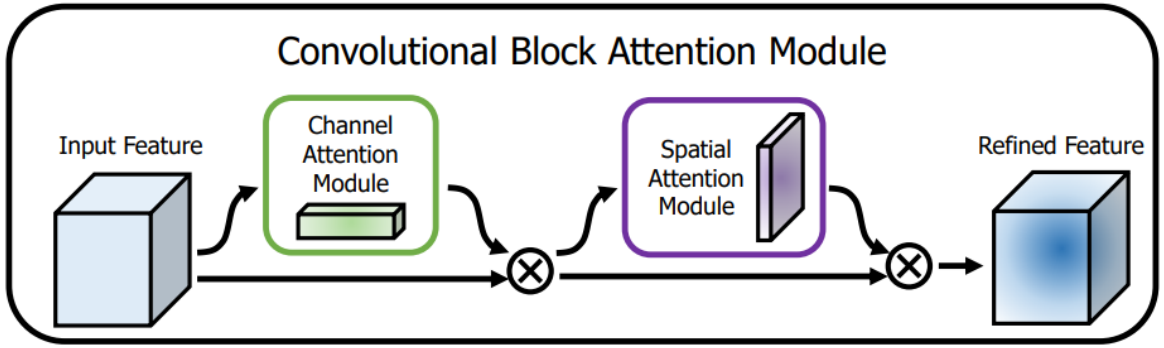
Attention is Not Just for Text

Spatial Attention

We chunk images spatially and implement an attention mechanism to allow a model to decide which portions of the image it should focus on.

Channel Attention

We implement an attention mechanism that allows the model to decide which feature maps are important for a given prediction task. You could, e.g., flatten each feature map into a vector and treat it just like word embeddings are handled in the QKV framework. You can similarly take 'patches' of input (like we do in pooling layers) and decide which to attend to in the same way.



Questions?